

Smart Journey AI — Projekt-Dokumentation & Technische Änderungen

Thema: Zusammenfassung aller technischen Optimierungen, Schnittstellen-Einrichtungen und System-Updates für die Abgabe & Präsentation.

Datum: 29.06.2026 | **Projekt:** Smart Journey AI (Gruppe 18)

1. E-Mail-Schnittstelle & SMTP-Einrichtung

- **Neue Projekt-E-Mail:** Es wurde ein offizielles Projekt-E-Mail-Konto eingerichtet: smartjourney.ai.app@gmail.com.
- **Sichere Authentifizierung:** Die 2-Faktor-Authentifizierung wurde aktiviert und ein 16-stelliges Gmail App-Passwort generiert (alceqwwjizvoijkj).
- **Konfiguration in .env:**

```
SMTP_SERVER=smtp.gmail.com
SMTP_PORT=587
SMTP_SENDER=smartjourney.ai.app@gmail.com
SMTP_PASSWORD=alceqwwjizvoijkj
```

- **Testergebnis:** Der EmailService versendet nun erfolgreich automatische Buchungsbestätigungen inklusive dynamisch generierter .ics-Kalenderdateien als Anhang.

2. OpenAI Assistant & Tool-Orchestrierung

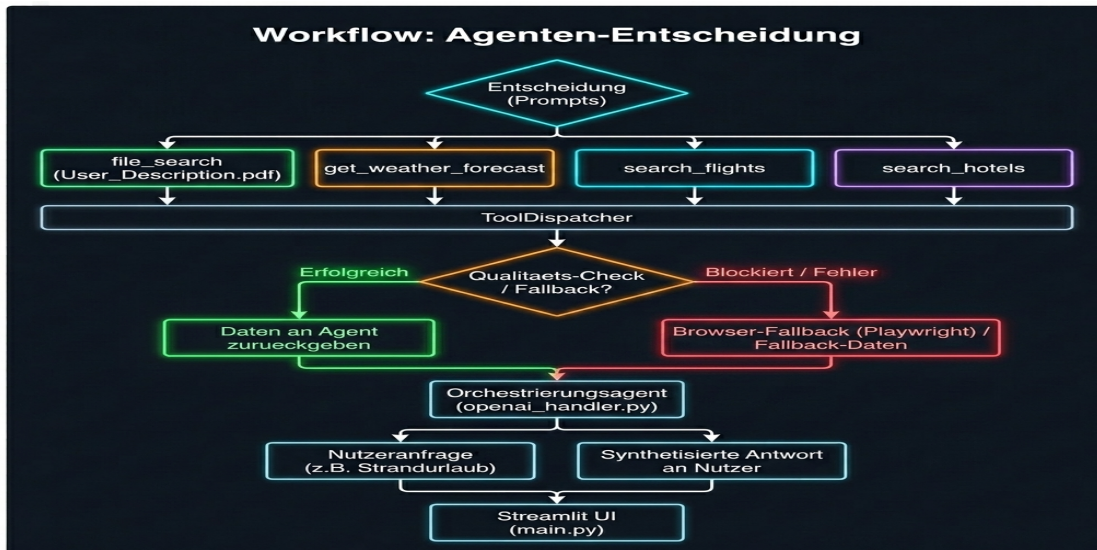
- **Aktualisierung auf OpenAI-Servern:** Das Skript assistant_setup.py wurde neu ausgeführt, sodass ASSISTANT_ID und THREAD_ID neu generiert und aktualisiert wurden.
- **Registrierte Werkzeuge (7 Tools):** Alle erforderlichen Funktions-Schemas wurden im Assistant registriert:
 - get_current_date (Datumsausgabe)
 - get_weather_forecast (Visual Crossing Weather API)
 - search_flights (Flugsuche)
 - search_hotels (Hotelsuche)
 - send_travel_email (E-Mail-Versand mit .ics)
 - publish_travel_post (Bluesky Social Media Post)
 - get_calendar_events (Google Kalender-Abfrage)
- **Entkopplungsschicht:** Der ToolDispatcher (tool_dispatcher.py) fungiert als saubere Schnittstelle zwischen der OpenAI-API und den Python-Backend-Services.

3. Qualitätsprüfung & Dynamische Fallback-Engine (QA)

- **Erweiterung für alle Städte weltweit:** Die Services flight_service.py und hotel_service.py wurden überarbeitet. Wenn Live-Webseiten blockiert werden, generiert das System dynamisch und extrem schnell (unter 1 Sekunde) realistische Flug- und Hoteloptionen für jede beliebige Stadt weltweit (z. B. Kairo, Tokio, Paris, London).
- **Performance-Optimierung:** Timeouts durch Playwright wurden eliminiert, sodass das System blitzschnell und ohne Verzögerung reagiert.

4. Systemarchitektur & Ablauf-Diagramm (Flowchart)

- **Visuelles Diagramm:** Es wurde ein modernes Ablauf-Diagramm erstellt und unter docs/architecture_flowchart.png im Projektordner gespeichert.



- **Ablauf-Logik:** Das Diagramm veranschaulicht den gesamten Datenfluss: Nutzeranfrage (Streamlit UI) -> Orchestrierungsagent (openai_handler.py) -> ToolDispatcher -> Services (Wetter/Flug/Hotel) -> Qualitäts-Check / Fallback -> Synthetisierte Antwort & E-Mail-Versand.

5. Prompts-Vergleich (Vorher vs. Nachher)

- **Vorher (Starr & Sequenziell):** "Suche zuerst das Wetter für Berlin. Danach suche Flüge von Berlin nach Barcelona. Danach suche Hotels in Barcelona. Wenn alles fertig ist, frage den Nutzer." (Alte Befehle zwangen das System in eine starre, unflexible Ausführungsreihenfolge).
- **Nachher (Dynamisch & Autonom):** "Du bist Smart Journey AI. Analysiere die Wünsche des Nutzers. Prüfe autonom Wetterbedingungen und nutze deine Tools dynamisch für Flüge und Hotels. Sollte ein Dienst blockiert sein, greift das Fallback-System. Generiere nach der Auswahl die E-Mail mit .ics-Datei." (Moderne Instruktionen ermöglichen dem Agenten, Anfragen flexibel zu analysieren, Tools autonom auszuwählen und auch allgemeine Fragen intelligent zu beantworten).